第4章 数据级并行（DLP）（向量体系结构与GPU）

4.1 数据级并行核心基础

4.1.1 数据级并行（DLP）定义与应用场景

1. 核心定义

数据级并行（Data-Level Parallelism，DLP）：**对大量独立数据执行相同操作/同种变换的并行计算模式**，核心是“单种操作、批量数据”，是高性能计算的核心并行方式之一。

2. 典型应用场景

科学计算：大规模矩阵运算、数值模拟、微分方程求解

图形图像：三维渲染、图像滤波、像素处理、视频编解码

通用并行计算：批量数据预处理、人工智能推理/训练

4.1.2 SPMD编程模型

1. SPMD定义

SPMD（Single Program Multiple Data，单程序多数据）：**所有并行单元执行同一份程序代码，各自处理不同的独立数据**，是适配数据级并行的核心编程模型。

2. SPMD在不同硬件上的执行方式

单核SISD架构：软件层面拆分批量数据，**将多数据（MD）拆分为多次单数据（SD）**，串行反复执行同一程序，无硬件并行，效率极低。 **耗时**

多核MIMD架构：硬件复制多套流水线，**将单程序（SP）指令多次重复分发**，多核同时执行、处理多组数据，实现并行，但存在严重的硬件冗余开销。 **耗电**

4.1.3 传统标量CPU的固有缺陷

传统标量CPU流水线（取指F、译码D、执行E）针对串行任务设计，在SPMD数据并行场景下存在致命短板：

指令开销冗余：**批量数据执行相同指令时，取指、译码操作被重复执行，逻辑复杂、耗时耗电；**

计算资源不足：单ALU单次仅能处理单个数据，批量数据处理需要大量循环迭代；

功耗与延迟瓶颈：多核并行会成倍增加控制逻辑开销

4.1.4 SIMD并行架构核心思想

SIMD（单指令多数据）是解决传统CPU冗余问题的专用硬件架构，核心设计理念：**精简前端、强化执行端**

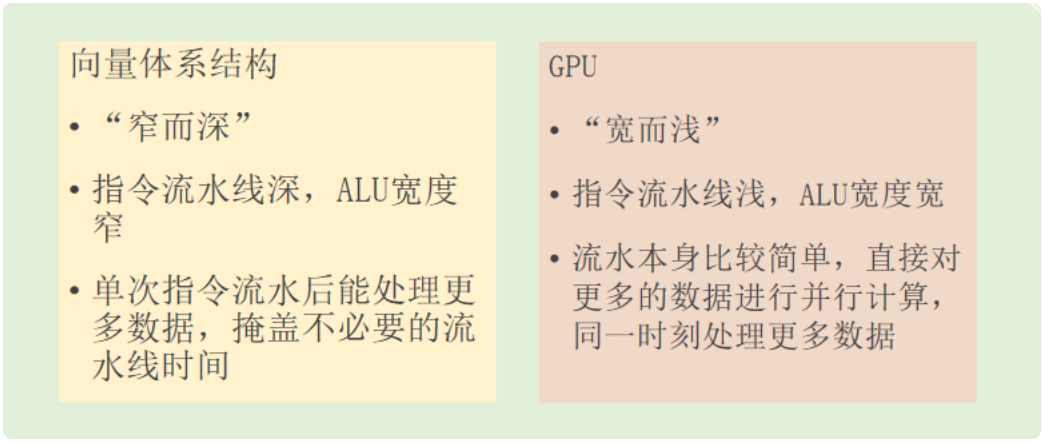
减少取指、译码单元数量：降低指令解析的时间、硬件、功耗开销，避免重复译码取指；

增设大量ALU执行单元：单条指令可驱动多个ALU，同时处理多组独立数据；

增加数据寄存器：一次存储更多数据，靠寄存器里的数据直接算，减少对主存（内存）的访问次数，从而降低整体的访存延迟。

4.1.5 向量体系结构与GPU的核心区别

二者均为SIMD思想的硬件落地，是数据级并行的两大核心架构，核心差异如下：



解释：

把向量处理器想象成一条**很长的传送带（流水线）**，用来给快递打包：

- 流水线步骤：装盒 → 贴标签 → 封胶带 → 扫描出库
- 标量 CPU 的做法：来 1 个快递，跑一遍完整流水线，4 个步骤都走一遍，再处理下一个。
- 向量处理器的做法：把**一整车快递**（比如 100 个）一次性放到流水线上：
  1. 前 4 拍：流水线正在“预热”，第一个快递走完 4 个步骤；
  2. 第 5 拍起：每 1 拍都能出来 1 个打包好的快递；
  3. 100 个快递，只花了 `4 + 99 = 103` 拍就全部处理完了。

4.2 向量体系结构（核心重难点）

4.2.1 三种向量计算方式（必考）

以运算公式 （向量长度为N）为例，三种计算方式的差异、开销及适用场景如下：

以计算 $D = A \times (B + C)$ 为例，A、B、C、D均为长度为N的向量

1. 横向计算（标量串行方式）

计算逻辑：按行逐个元素计算，依次执行 ，通过循环迭代实现。

核心开销

数据相关：N次（每个元素计算均存在数据依赖）；

功能切换：2N次（每个元素需切换加法、乘法两次运算）；

缺陷：向量长度N越大，开销线性增长，完全不适合向量并行处理，仅为标量CPU的通用计算方式。

向量的计算方式 – 1. 横向计算

- 向量计算是按行的方式从左到右横向地进行

  先计算： $d_1 \leftarrow a_1 \times (b_1 + c_1)$

  再计算： $d_2 \leftarrow a_2 \times (b_2 + c_2)$

  ……

  最后计算： $d_N \leftarrow a_N \times (b_N + c_N)$
- 组成循环程序进行处理

  $k_i \leftarrow b_i + c_i$

  $d_i \leftarrow k_i \times a_i$

  数据相关：N 次　功能切换：$2N$ 次
- 这种计算方式是在标量处理器上对向量的一般计算方式，不适合于向量处理机的并行处理。

2. 纵向计算（向量最优基础方式）

计算逻辑：按列批量计算，先完成**全部元素加法**，再完成**全部元素乘法**，两条完整向量指令完成计算。

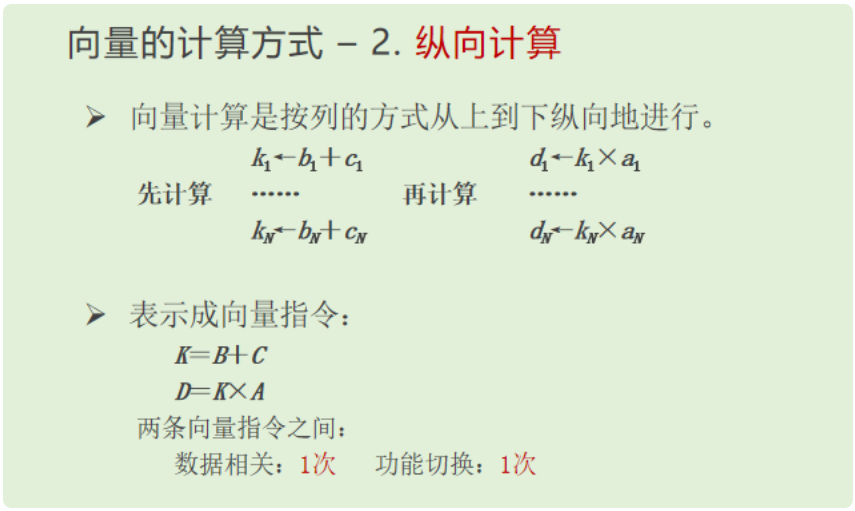
核心开销

数据相关：1次（仅两条向量指令间存在**全局数据依赖**）；

功能切换：1次（仅加法、乘法一次切换）；

优势：极大减少指令切换和数据相关开销，适配向量处理器并行特性；

缺点：每次计算都需要访问到向量中的全部元素。



3. 纵横（分组）计算（工程实际方式）

设计背景：**计算机存储层次化，寄存器容量有限，无法加载无限长度的向量N，因此结合横向、纵向优势采用分组计算。**

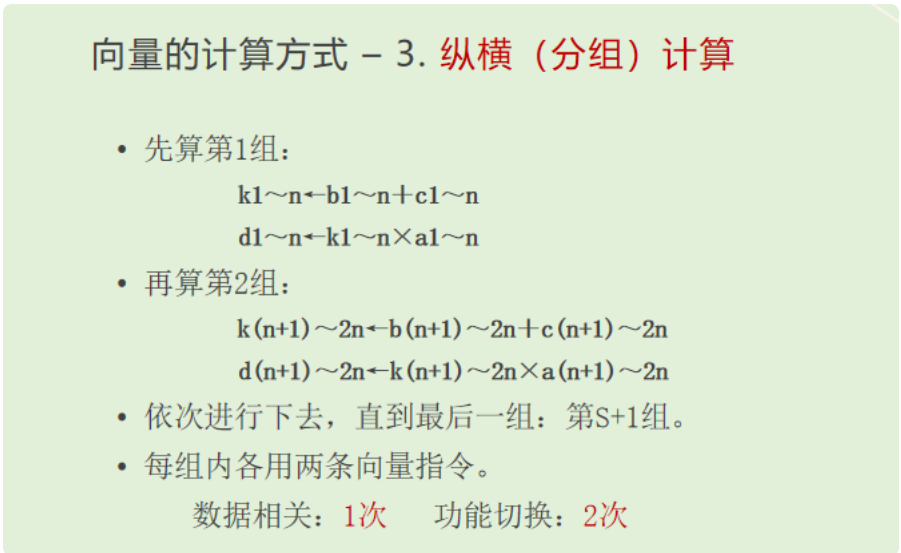
计算逻辑

将长向量拆分：（N=总向量长度，S=完整组数，n=单组最大长度，r=余数），余数单独作为一组，总组数S+1。

N=S×n＋r

组内采用纵向计算，组间串行迭代，依次处理每一组数据。

核心开销：**每组数据相关1次、功能切换2次**，平衡了寄存器容量限制和计算开销，是硬件实际运行的主流方式。



4.2.2 向量体系结构核心特性

大容量顺序寄存器堆：批量加载向量元素，支撑纵向批量计算，替代频繁访存；

全流水化访存（LD/ST）：向量体系结构从内存中收集散落的数据，将其放入寄存器堆中，并对寄存器堆中的数据们进行操作，然后将这些结果放回内存（一次传输一组数据，LD/ST流水化）

【能够掩盖访存延迟并充分利用内存带宽；】

单指令批量操作：一条向量指令可操作整个向量的所有元素，实现数据级并行；

低功耗低复杂度：对比超标量乱序CPU，向量架构扩展简单，不会大幅提升硬件复杂度和功耗，规避功耗墙。

4.2.3 典型实例：VMIPS架构

VMIPS = 基础标量MIPS架构 + 逻辑向量扩展模块，兼容标量指令，支持向量并行计算。

1. 核心硬件组成

向量寄存器（VR）：8个向量寄存器，每个寄存器64bit*64元素，配备充足读写端口，支持批量数据读写；

向量功能单元：全流水化设计，每个时钟周期可完成1次运算，内置冒险检测单元（检测结构冒险、数据冒险）；

向量LD/ST单元：全流水化，单周期传输1个字，同时支持标量、向量访存；

标量寄存器：32个通用寄存器+32个浮点寄存器，负责提供访存地址、辅助计算数据。

2. 向量专用特殊寄存器

向量长度寄存器VL：64位寄存器，每一位对应一个向量寄存器单元，作用是**将软件层程序中实际向量长度N与硬件层向量寄存器中的元素数目64相适配**

核心定位：控制向量运算的“有效长度”

你可以把它理解成一个 **“长度控制器”**。

1. **硬件背景**
   向量寄存器的物理长度是固定的（比如这里是 64 个元素），但程序里的向量长度 N 可以是任意数（比如 10、30、100...）。
   VL 的作用就是让硬件知道：**这一轮计算，我只用到前 N 个元素，后面的硬件别碰。**
2. **怎么工作?**
   - 它是一个 64 位的寄存器，每一位对应向量寄存器里的一个元素。
   - 比如程序里 N=10，VL 的低 10 位设为 1，高 54 位设为 0。
   - 硬件在执行向量加法、乘法、访存（ld/st）时，只会处理前 10 个元素，剩下的自动忽略。

向量屏蔽寄存器VM：用于向量长度不足64、或条件分支场景，屏蔽无效元素，即使存在大量屏蔽位，运算速度仍远优于标量计算。

核心定位：控制向量运算的“有效元素”

你可以把它理解成一个 **“开关掩码”**，决定哪些元素参与运算，哪些不参与。

1. **两种典型使用场景**
   - **场景 1：向量长度小于 64**
     比如 N=10，你可以用 VM 的低 10 位设为 1，实现和 VL 类似的效果（但 VL 是更高效的专用控制）。
   - **场景 2：条件分支 / 不规则计算**
     比如 `if (A[i] > 0) B[i] = A[i] * 2`，你需要对部分元素执行操作，这时：
     - 把满足条件的元素对应的 VM 位设为 1；
     - 向量指令执行时，只对 VM=1 的元素进行运算，VM=0 的元素保持不变。
2. **关键特点：即使大部分是 0，依然比标量快**
   哪怕 VM 里大部分都是 0，硬件依然是用**一条向量指令**处理整个向量，而不是拆成标量循环。

表格

维度	向量长度寄存器 VL	向量屏蔽寄存器 VM
控制对象	控制**连续的前 N 个元素**	控制**任意位置的零散元素**
适用场景	规则的、固定长度的向量运算	带条件分支、不规则的向量运算
控制方式	按长度控制，只指定 N	按掩码控制，每一位单独开关
优先级	决定了本次运算的最大长度	在 VL 指定的长度内，进一步筛选元素

3. VMIPS指令格式与代码对比

指令规则：VMIPS指令=MIPS指令+Op1类型+Op2类型.精度

示例：标量指令add.d → 向量指令addvv.d(向量+向量)、addvs.d(向量+标量)

核心优势：不仅精简代码行数，更关键是**消除大量循环迭代、指令译码、访存冗余开销**，从硬件层面实现并行加速。

4.2.4 向量体系核心概念

循环间相关：**循环的多轮迭代之间存在数据依赖，前一轮计算结果影响后一轮执行；无相关的循环可向量化。**

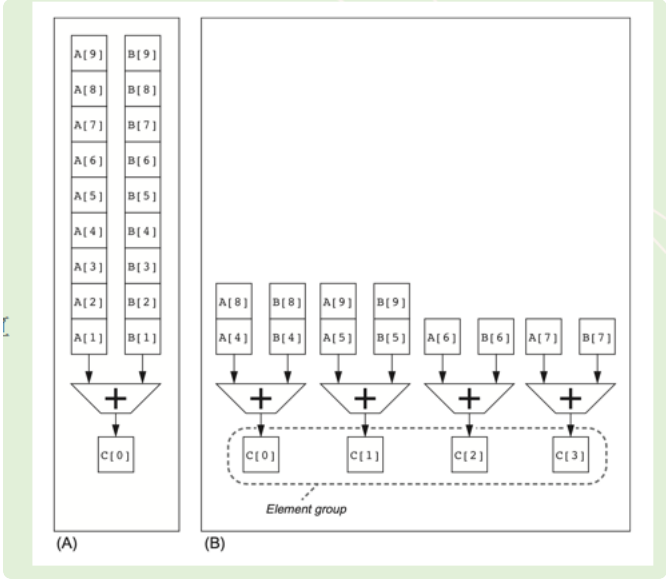
可向量化：若循环无循环间相关，编译器可将标量循环代码编译为向量指令，实现并行加速。

指令编队（convoy）：**无结构冒险、可并行执行的一组向量指令集合，同一编队指令可同时启动执行。**

4.2.5 向量四大性能优化技术

1. 多车道技术

核心原理：**增加ALU功能单元数量**，类比“单车道扩为多车道”，单周期可并行处理更多向量元素，直接提升计算吞吐量，是硬件层面的并行加速手段。



2. 链接技术

定义：**两条存在写后读数据相关的向量指令，若无寄存器冲突、功能部件冲突，可将流水线头尾对接，形成联合流水线。**

1. **结果寄存器冲突**（最严重）
   两条指令要把结果写到**同一个向量寄存器**
   例：
   V2 = V0+V1
   V2 = V3×V4
   → 都要写 V2，**冲突！不能链接**
2. **源寄存器冲突**
   两条指令同时读**同一个源寄存器**，但读取节奏 / 长度不一样，硬件没法同时供数
   例：
   V3 = V1+V2
   V6 = V1×V4
   → 都读 V1，但硬件不能同时给两条指令用
   → 冲突！不能链接
3. **源 / 目的重叠乱套**
   一条指令的源，刚好是另一条指令的目的，但不是正常写后读，而是乱重叠
   → 也算冲突

核心逻辑：无链接情况下，后面的功能需要等到前一个功能的n个结果都产生才能开始；而链接情况下，后面的功能只需要等到前一个功能的第一个结果产生就可以开始，即向量数据的生产与向量数据的消费进行延迟的重叠。

链接实质：把流水线定向的思想引入到向量执行过程，对两条流水线进行联合控制，没有改变寄存器和运算电路。

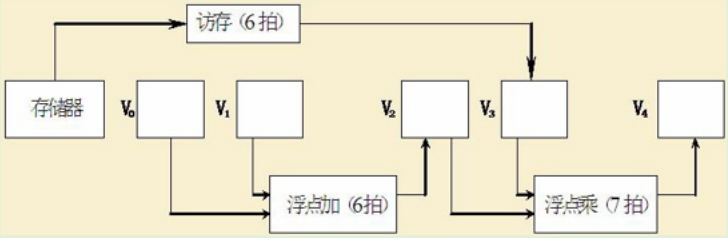
例题

假设功能单元的时间开销为：浮点加减(6 cycle)，浮点乘法(7 cycle)，浮点存储操作(6 cycle)。为了同步要求，将向量元素送往功能部件，以及把结果存入向量寄存器需要一拍时间，从存储器中把数据送入访存功能部件也需要一拍时间。为以下程序画出链接示意图，并分析非链接执行和链接执行两种情况下的执行时间。

$V_3 \leftarrow$ 存储器 // 访存取向量A

$V_2 \leftarrow V_0 + V_1$ // 向量B和向量C进行浮点加

$V_4 \leftarrow V_2 \times V_3$ // 浮点乘，结果存入V_4



1. 三条指令全部用串行方法执行，则执行时间为：

$$[(1+6+1)+N-1]+[(1+6+1)+N-1]+[(1+7+1)+N-1] = 3N+22 \text{ (拍)}$$

2. 前两条指令并行执行，然后再串行执行第3条指令，则执行时间为：

$$[(1+6+1)+N-1]+[(1+7+1)+N-1] = 2N+15 \text{ (拍)}$$

3. 前两条指令并行执行，并与第3条指令链接执行。

- 从访存开始到把第一个结果元素存入V_4所需的拍数（亦称为链接流水线的建立时间）为：

$$[(1+6+1)]+[(1+7+1)] = 17 \text{ (拍)}$$

- 生成所有结果的执行时间为：

$$17+(N-1) = N+16 \text{ (拍)}$$ 解毕。

进行向量链接的要求：

要求一：当一条向量指令的两个源操作数分别是两条先行指令的结果寄存器时，要求先行的两条指令产生运算结果的时间必须相等，即要求有关功能部件的通过时间相等。

> 一条向量指令双源分别来自两条先行向量结果
> **必须两条先行指令通过时间相等**
> 目的：**两条先行指令的元素流严格对齐、节拍同步流出**

两种区别

1. 通过时间相等（理想可链接）

指令 A：8 拍出第一个元素，之后**每 1 拍出 1 个**
指令 B：8 拍出第一个元素，之后**每 1 拍出 1 个**
结果：
A_0、B_0 同拍出
A_1、B_1 同拍出
A_2、B_2 同拍出
从头到尾所有元素一一严格对齐
后面乘指令直接无缝对接，**全程无气泡**

要求二：要进行链接执行的向量指令的向量长度必须相等，否则无法进行链接。

一次链接行为通常仅发生在分组内部，即不对整个N进行链接，而对个分组内的n个向量元素的计算过程进行链接

3. 编队技术

- 几条能在同一个时钟周期内一起开始执行的向量指令集合称为一个编队；
- 同一个编队中的向量指令之间
  - 不存在结构冲突；
  - 不存在数据冲突；
  - 存在数据冲突，但是可以链接。

例题

编队技术例题

假设每种向量功能部件只有一个，下面一组向量指令，在不使用链接技术和使用链接技术的情况下如何编队？

```
LV      V1，Rx      //从存储器中载入向量 Rx
MULTSV  V2，R0，V1  //向量 V1 和标量寄存器 R0 中的内容相乘
LV      V3，Ry      //从存储器中载入向量 Ry
ADDV    V4，V2，V3  //向量 V2 和 V3 相加，结果保存在向量寄存器 V4 中
SV      Ry，V4      //将向量寄存器 V4 中的结果保存在存储器向量 Ry 中
```

解：不使用链接技术分为4个编队
- 第一编队：LV
- 第二编队：MULTSV； LV
- 第三编队：ADDV
- 第四编队：SV

使用链接技术分为3个编队
- 第一编队：LV；MULTSV；
- 第二编队：LV；ADDV；
- 第三编队：SV

注意： 访存部件（Load/Store 共享一个）同一时间只能处理一条访存指令。

4. 分段开采技术

应用场景：软件向量长度N > 硬件向量寄存器最大长度n。

- 当向量的长度N大于向量寄存器的长度n时，必须把长向量N分成长度固定为n的段，然后循环分段处理，每一次循环只处理一个向量段。这种技术称为分段开采技术。
- 由系统硬件和编译软件合作完成控制，对程序员是透明的。

4.2.6 向量体系性能影响因素

- 操作数向量的长度
- 向量启动时间
- 操作之间的数据相关，是否采用链接
- 操作之间的结构性相关，发射限制，车道数量，是否采用编队